Reinforcement Learning

Lecture 1 — Multi-armed Bandits: Evaluative Feedback

Przemysław Witaszczyk

Faculty of Physics, Astronomy and Applied Computer Science
Jagiellonian University

2026/2027

Lecture plan overview

- L1 Multi-armed bandits: evaluative feedback
- L2 Finite Markov decision processes
- L3 Dynamic programming: planning with a known model
- L4 Monte Carlo methods
- L5 Temporal-difference learning
- L6 n-step bootstrapping & eligibility traces
- L7 Planning and learning combined (Dyna, MCTS)
- L8 On-policy function approximation
- L9 Value-based deep RL: the DQN family
- L10 Policy-gradient methods: REINFORCE
- L11 Actor–critic & advanced policy methods
- L12 Model-based RL, frontiers, synthesis
- L13 AlphaZero — a capstone case study

Sources: Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.) — the theoretical basis — and Morales, *Grokking Deep Reinforcement Learning* — the hands-on companion.

The domains of machine learning

Reinforcement learning is contrasted here with the other kinds of machine learning — they differ mainly in *what signal the learner is given*:

- **Supervised learning:** every example comes with the *correct answer* (a label); regressors and classifiers learn a map input $\rightarrow$ label.
- **Unsupervised learning:** no labels — the learner finds *structure* (clustering, dimensionality reduction).
- **Self-/semi-supervised learning:** labels are *manufactured from the data itself* (predict a masked word, reconstruct an input) — autoencoders, LLMs.
- **Reinforcement learning:** no labels and no fixed dataset — the learner *acts*, and the only feedback is a scalar *reward*.

RL differs from all three: its data is *generated by its own choices*. It still *uses* the other domains as tools — a neural network can approximate the value or policy when states are too many to tabulate (**Deep RL**, treated in the later lectures).

Special features of RL: what makes it hard (1)

Each of these sets RL apart from supervised learning — and each is a difficulty the methods of this course address:

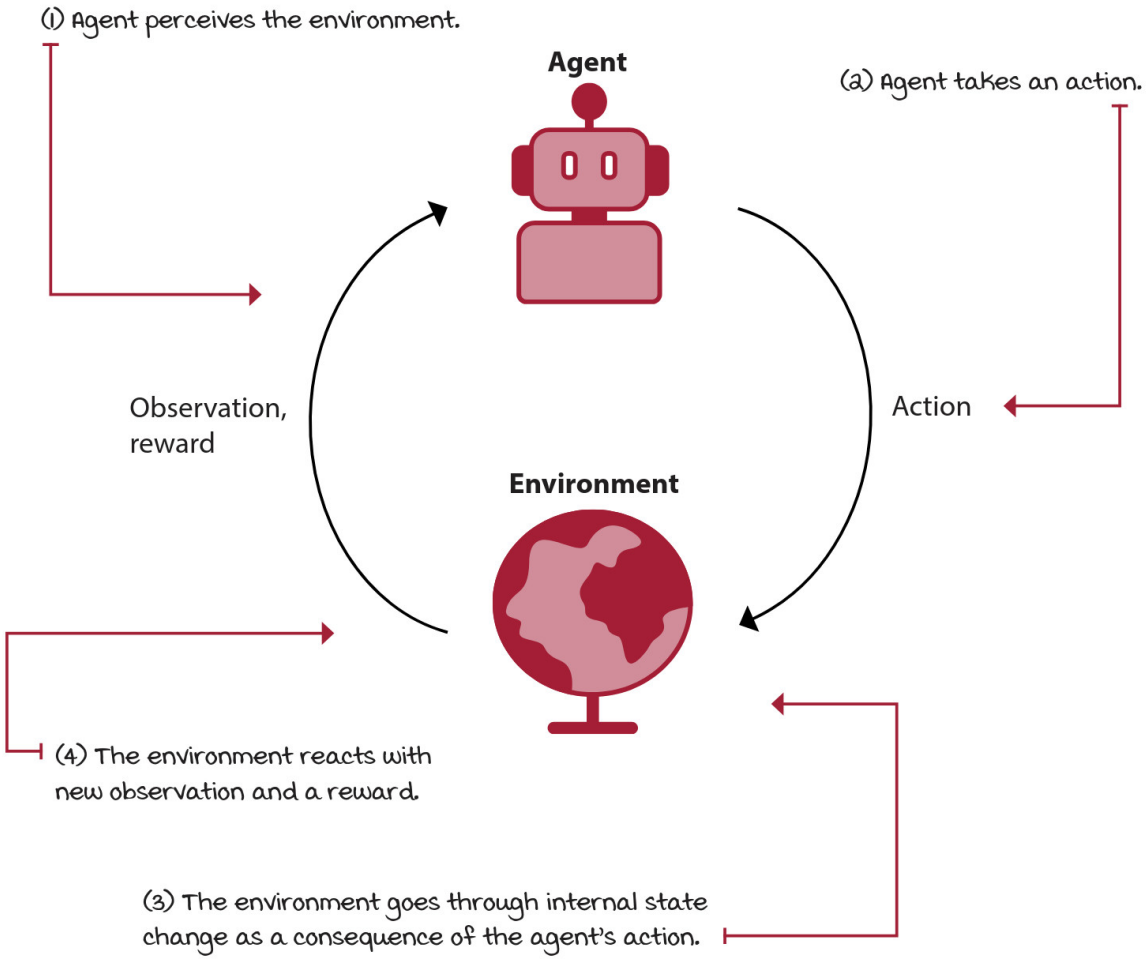
- **Sequential problem.** Samples are *causally linked*: the action taken now changes the state observed next. Data points are not independent draws — they form a trajectory.
- **Delayed success signal.** A reward may be received many steps after the action that produced it. Determining *which* past action is responsible for the reward is the **credit-assignment problem**.
- **Interaction with a (stochastic) environment.** There is no fixed dataset; the agent must *act* to generate its own data, and the environment may respond randomly.
- **Partial observability.** The agent often sees only an imperfect *observation*, not the true underlying state.

Special features of RL: what makes it hard (2)

- **Sampled experience.** The agent visits only a *small subset* of all possible states — it must generalize from the few it has seen to the many it has not.
- **Broken IID / stationarity.** Supervised learning assumes data is **independent, identically distributed** (IID) and drawn from a fixed distribution. In RL both fail: samples are correlated (sequential), and the distribution shifts as the agent's policy changes.
- **No clear success criterion.** There is no ground-truth label; behaviour is shaped entirely through the reward signal, which *we* must design — "reward engineering" — and a misspecified reward yields unintended behaviour.

Nearly every technique in the course addresses one of these seven features. Each of the seven is referenced again at the point where it recurs.

Reinforcement Learning Interaction Cycle



The defining loop: the **agent** receives an **observation** and issues an **action**; the **environment** responds with a **reward** and a new observation; repeat. Learning happens *inside* this loop — no teacher, only this stream of interaction.

The components of an RL system

The diagram supplies most of the key components used to frame the RL problem (randomness is implicit):

- **Agent** — the learner and decision-maker. It acts according to its **policy**: a rule mapping what it observes to what it does.
- **Environment** — everything outside the agent; the system the agent acts in and which responds to those actions.
- **Actions** — the agent's means of influencing the environment.
- **Observations** — what the environment reveals back to the agent (possibly a partial view of its true state).
- **Rewards** — the scalar feedback signal; the *only* measure of success available to the agent.
- **Randomness** — environments are in general *stochastic*: the same action in the same state need not give the same outcome.

These six components suffice to state the whole RL problem; each is made precise in the lectures that follow.

A roadmap for this lecture and the next

All the core objects of reinforcement learning are introduced in the simplest possible setting — the **multi-armed bandit** — and each is then *lifted* to the full problem.

Today (bandit, "0-dim" RL)	$\longrightarrow$	**Next lecture (MDP)**
action value $q_*(a)$	$\longrightarrow$	$q_\pi(s, a)$
estimate $Q_t(a)$	$\longrightarrow$	$Q(s, a),\ V(s)$
single reward R_t	$\longrightarrow$	return $G_t = \sum_k \gamma^k R_{t+k+1}$
(no state)	$\longrightarrow$	state S_t, transition $p(s' \mid s, a)$
explore vs. exploit (one slot)	$\longrightarrow$	explore vs. exploit (a trajectory)

The bandit has *no state and no sequencing*, so every equation appears in its simplest form. Once the bandit is understood here, the MDP is "just" the same ideas with states added.

What makes RL different: evaluative feedback

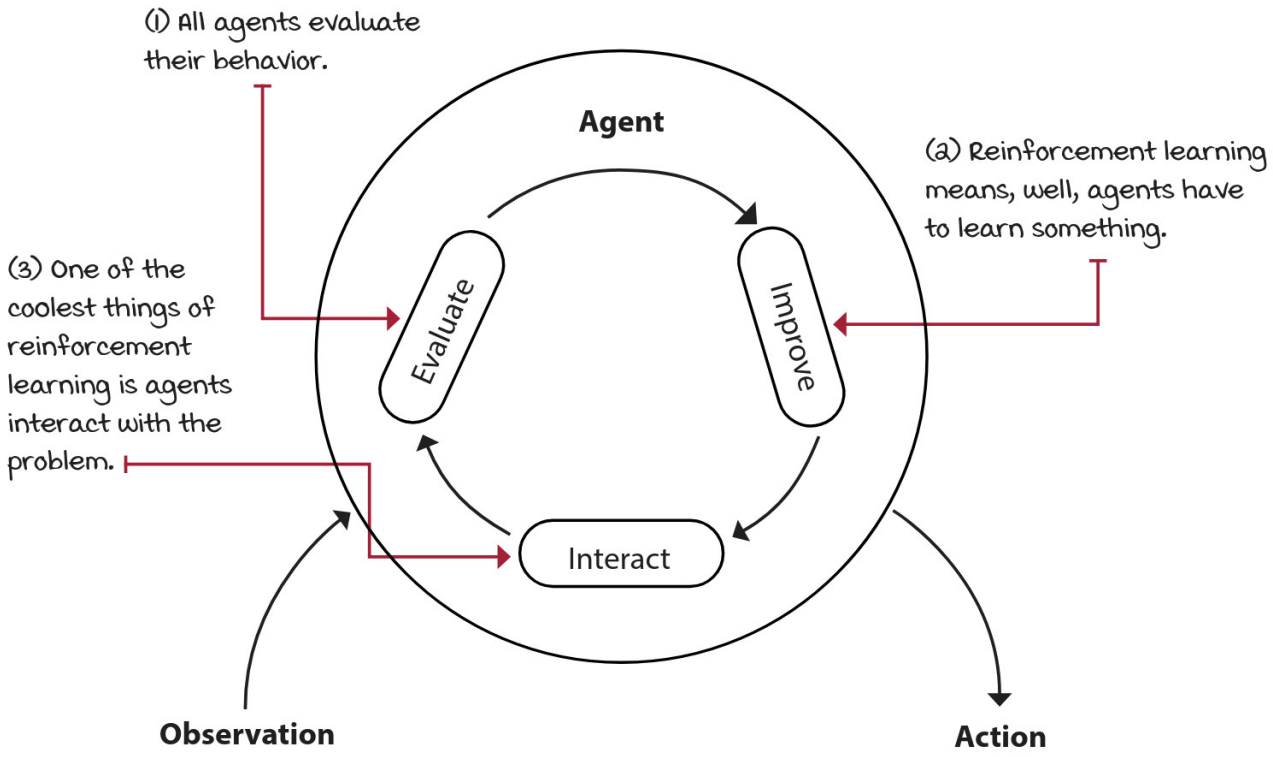
The single most distinguishing feature of reinforcement learning is the *kind of feedback* it uses.

- **Instructive feedback** (supervised learning): indicates the *correct* action, independently of the action taken.
- **Evaluative feedback** (RL): indicates only *how good* the action taken was — not whether it was the best possible.

Because evaluative feedback depends entirely on the action taken, the agent must **actively explore** to discover good behavior. This is the origin of the exploration–exploitation problem.

The bandit isolates this evaluative aspect in a nonassociative setting: there is only one situation to act in. (Sutton & Barto, Ch. 2)

The agent's inner loop: evaluate, improve, interact



Even in the bandit, the agent repeatedly *interacts* (pulls an arm), *evaluates* the reward, and *improves* its estimates. (Morales, Ch. 2)

The k-armed bandit problem: formal statement

Definition

A k-**armed bandit** is a finite action set $\mathcal{A} = \{1, \dots, k\}$ together with a family of *fixed, unknown* reward distributions $\{\nu_a\}_{a \in \mathcal{A}}$ on $\mathbb{R}$. At each step $t = 1, 2, \dots$ the agent selects an action $A_t \in \mathcal{A}$ and observes a reward

$$R_t \sim \nu_{A_t}, \qquad \text{independently of the past given } A_t.$$

- Two standard instances: a slot machine ("one-armed bandit") with k levers, and a doctor choosing among k treatments.
- The reward depends *only* on the chosen action, not on any state or on previous actions — the task is **nonassociative** and **memoryless**.
- The distributions ν_a are **stationary**: they do not change over time (relaxed under "Nonstationary problems" below).

(Sutton & Barto, §2.1)

Action values, optimality, and the learning goal

The **value** of an action is its mean reward,

$$q_*(a) \doteq \mathbb{E}[\, R_t \mid A_t = a \,] = \int_{\mathbb{R}} r \, \mathrm{d}\nu_a(r),$$

(the symbol "$\doteq$" means "is *defined* as"), and the **optimal value** and an **optimal action** are

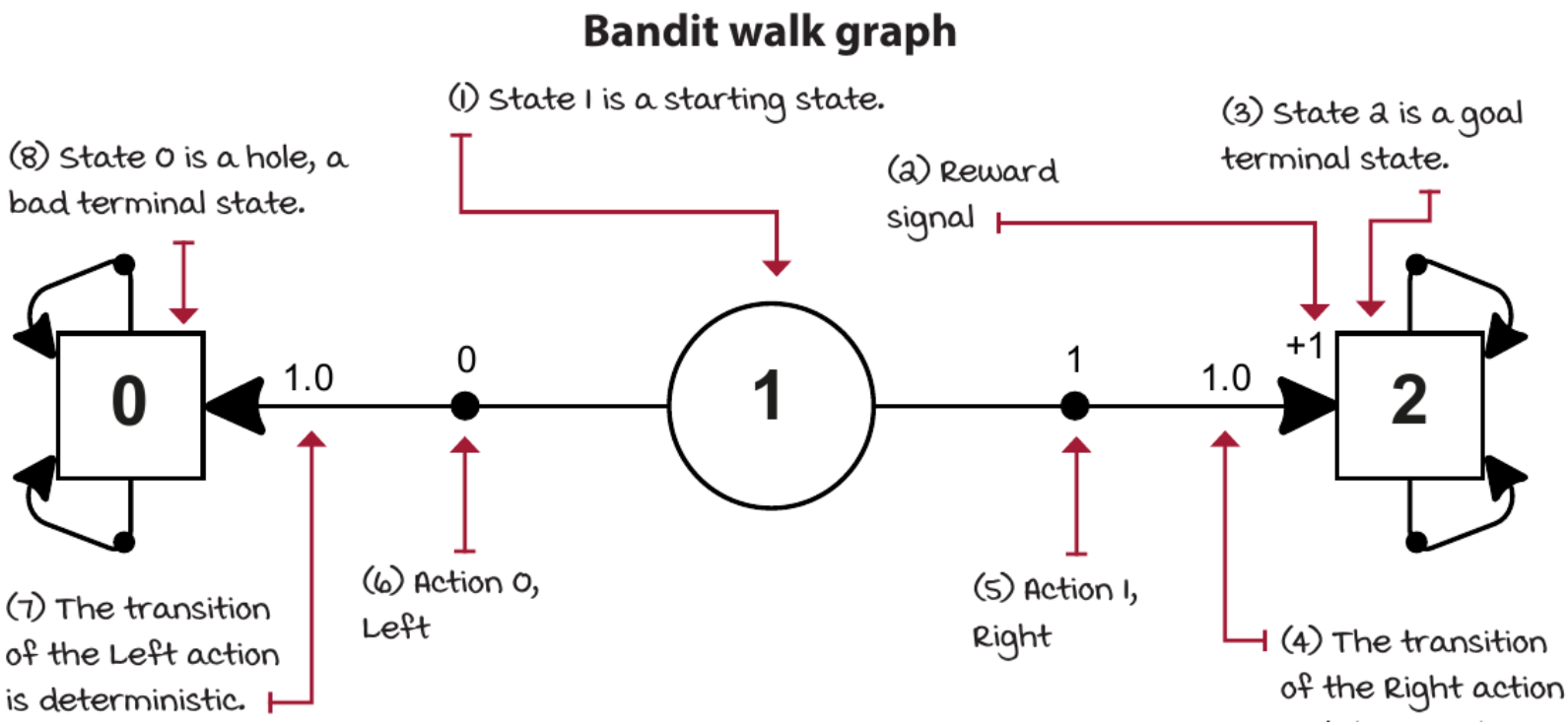
$$v_* \doteq \max_{a \in \mathcal{A}} q_*(a), \qquad a_* \in \arg\max_{a \in \mathcal{A}} q_*(a).$$

- If the $q_*(a)$ were known, the problem would require no learning: always play a_*. The difficulty is that they must be *estimated* from sampled rewards.
- **Objective.** Maximize the expected cumulative reward $\mathbb{E}[\sum_{t=1}^{T} R_t]$, equivalently minimize the (expected) **regret**

$$\mathcal{L}_T \doteq T\, v_* - \mathbb{E}\left[\sum_{t=1}^{T} R_t\right] = \sum_{t=1}^{T} \mathbb{E}[\, v_* - q_*(A_t)\,].$$

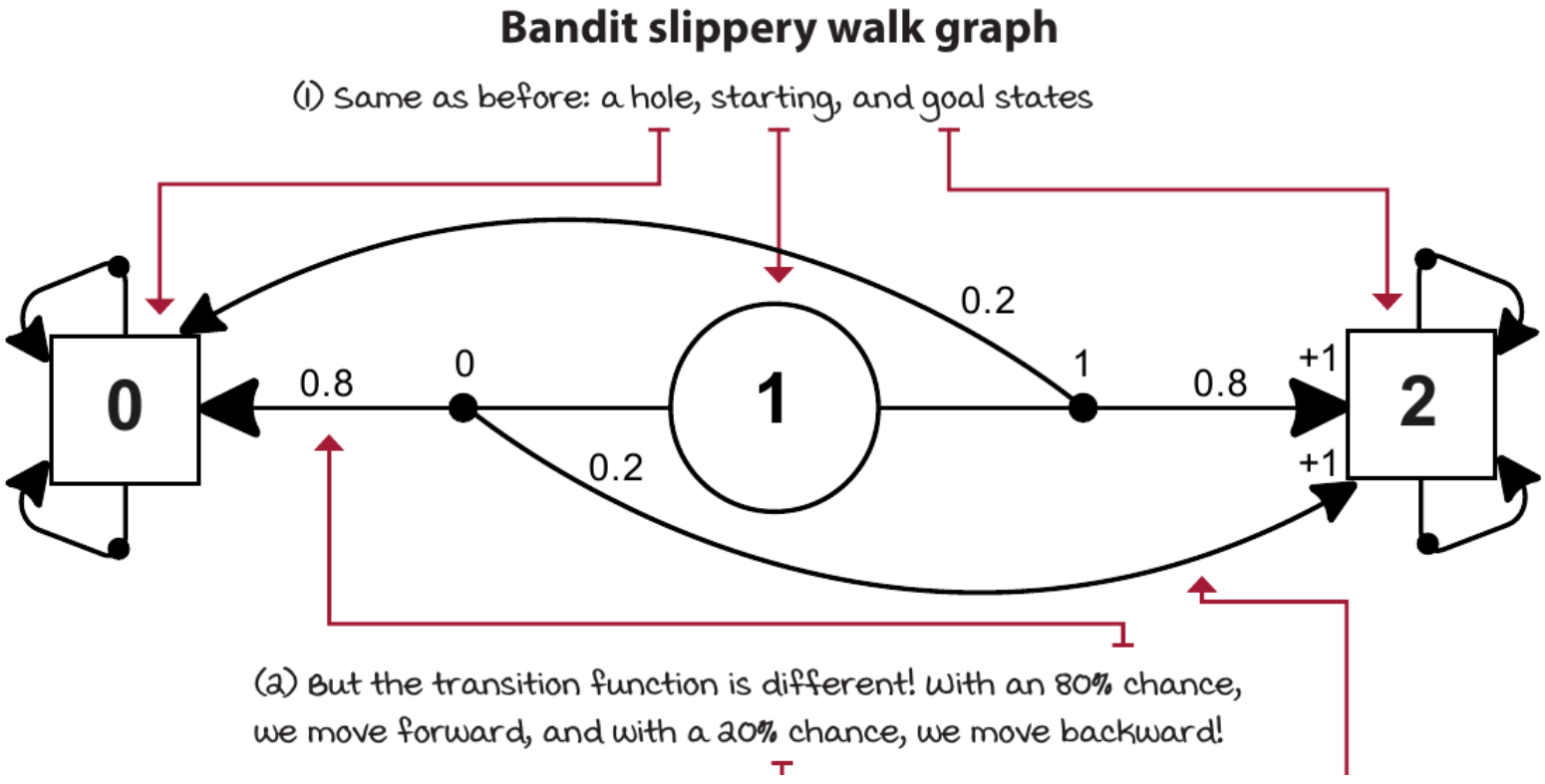
(Sutton & Barto, §2.1; v_*, regret: Morales, Ch. 4)

A concrete bandit-as-MDP: the Bandit Walk (BW)



Deterministic 3-state walk. State 1 is the start; state 0 is a bad terminal (hole), state 2 a good terminal (goal, $+1$). One interaction ends the episode — a *greedy horizon*. This is exactly a 2-armed bandit drawn as a graph. (Morales, Ch. 2)

Adding stochasticity: the Bandit Slippery Walk (BSW)



Same states, but the transition is **stochastic**: the intended move succeeds with prob. 0.8 and slips backward with prob. 0.2.

Read as a 2-armed bandit, the BSW gives each arm a *random* reward $R_t \in \{0, 1\}$ — i.e. ν_a is a Bernoulli law with

$$q_*(\text{Right}) = \Pr\{R{=}1 \mid \text{Right}\} = 0.8, \qquad q_*(\text{Left}) = 0.2,$$

so $a_* = \text{Right}$, $v_* = 0.8$. This is exactly the evaluative-feedback setting Sutton studies.

(Morales, Ch. 2, 4)

The exploration–exploitation dilemma

We keep estimates $Q_t(a) \approx q_*(a)$. At any step, an action attaining $\max_a Q_t(a)$ is a **greedy** action.

- Selecting a greedy action is **exploiting** current knowledge — it maximizes expected reward *on this one step*.
- Selecting a non-greedy action is **exploring** — it may yield greater total reward *in the long run* by sharpening an uncertain estimate.
- A single action selection cannot do both — hence the "conflict".

Formally, regret $\mathcal{L}_T = \sum_t \mathbb{E}[v_* - q_*(A_t)]$ accumulates whenever $A_t \neq a_*$; a purely greedy agent that locks onto a suboptimal arm incurs *linear* regret $\mathcal{L}_T = \Theta(T)$. The aim is *sublinear* regret, $\mathcal{L}_T / T \to 0$. This is a distinctive RL challenge (recall: *delayed* success signal, *sampled* experience).

Action-value methods: the sample average

A natural estimate of $q_*(a)$ is the average of rewards received so far when a was taken:

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}}. \tag{2.1}$$

- $\mathbf{1}_{\text{predicate}}$ is 1 if true, 0 otherwise.
- If the denominator is 0, define $Q_t(a)$ as a default (e.g. 0).
- By the law of large numbers, $Q_t(a) \to q_*(a)$ as the count $\to \infty$.

Greedy and ε-greedy action selection

The **greedy** rule always exploits:

$$A_t \doteq \arg\max_a Q_t(a). \tag{2.2}$$

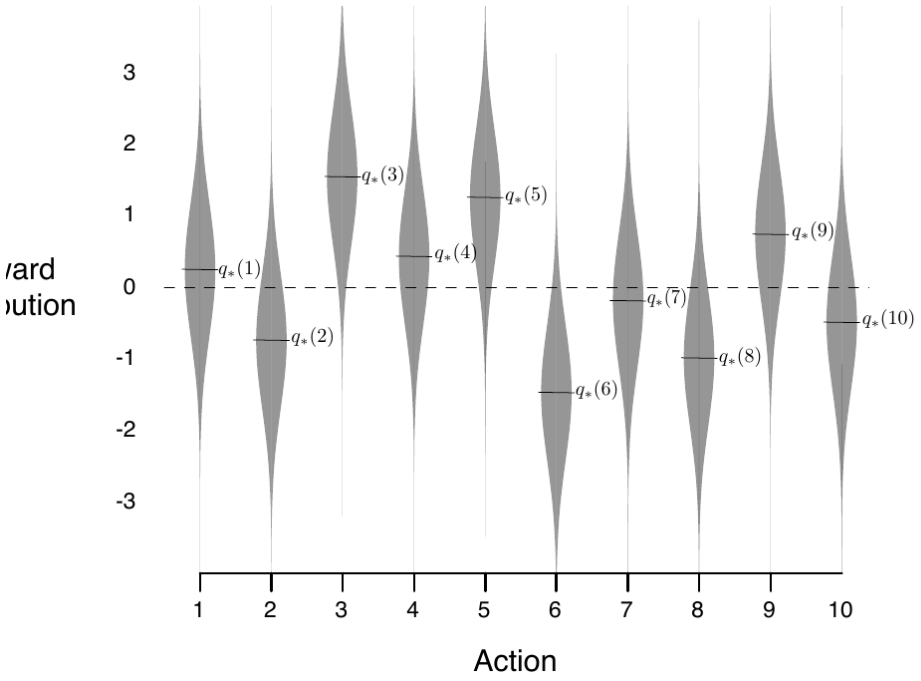
The ε-**greedy** rule explores occasionally:

$$A_t = \begin{cases} \arg\max_a Q_t(a) & \text{with probability } 1-\varepsilon, \\ \text{a random action} & \text{with probability } \varepsilon. \end{cases}$$

Why it works (asymptotically): every action is sampled infinitely often, so all

$Q_t(a) \to q_*(a)$, and the probability of selecting the optimal action converges to more than $1-\varepsilon$.

The 10-armed testbed



A controlled benchmark with $k = 10$:

$$q_*(a) \sim \mathcal{N}(0, 1), \quad a = 1, \ldots, 10,$$

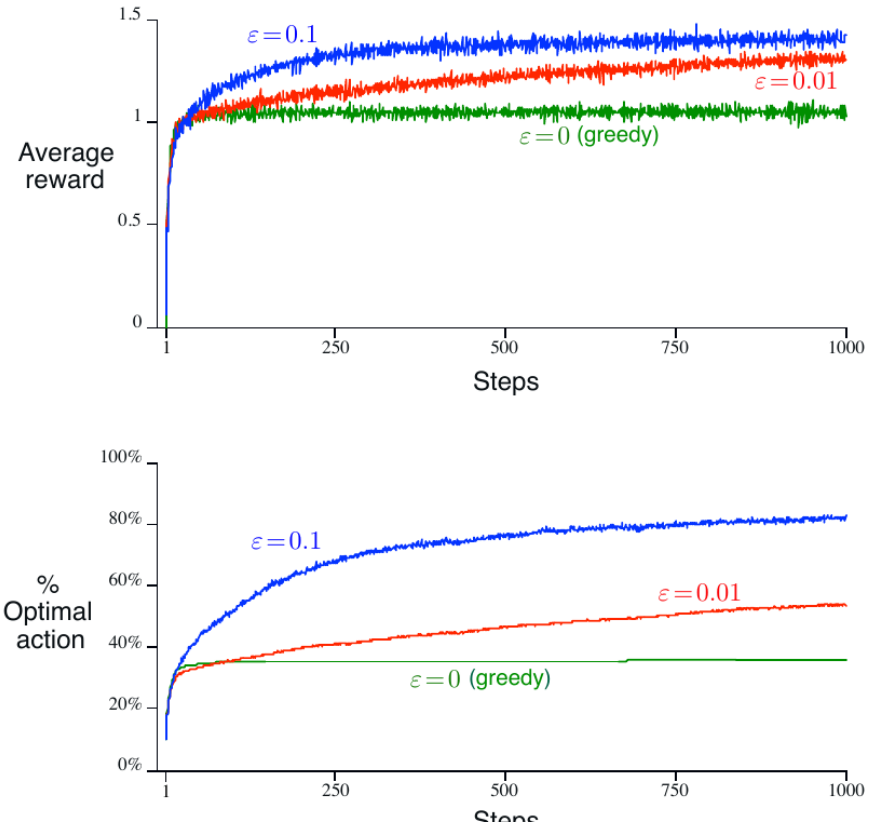
$$R_t \mid A_t \sim \mathcal{N}\big(q_*(A_t), 1\big).$$

- One *run* = 1000 steps on one such problem.
- Average over **2000** independent runs (each a fresh $\{q_*(a)\}$) to estimate *expected* behavior.
- Two metrics: average reward, and % of steps the optimal arm a_* is chosen.

(Sutton & Barto, §2.3, Fig. 2.1)

ε-greedy on the testbed: the effect of exploration

estimates using the sample-average technique (with an initial estimate of 0). Th
graph shows the increase in expected reward with experience. The greedy
improved slightly faster than the other methods at the very beginning, but then
off at a lower level. It achieved a reward-per-step of only about 1, compared with
possible of about 1.54 on this testbed. The greedy method performed significantl
in the long run because it often got stuck performing suboptimal actions. The lowe



Both $\varepsilon = 0.1$ and $\varepsilon = 0.01$ reach higher average reward and a higher optimal-action rate than the pure greedy method ($\varepsilon = 0$), which often gets stuck on suboptimal actions.

(Sutton & Barto, Fig. 2.2)

Incremental implementation: the problem

Consider a *single* action. Let R_i be the reward received after its i-th selection, and Q_n the estimate after it has been chosen $n-1$ times — the sample average

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}.$$

- **Naive implementation:** store every reward and recompute the sum on demand.
- Then *memory* and *per-step computation grow without bound* as more rewards arrive — each new reward needs more storage and a longer sum.

Goal. The update must process each new reward with *constant* memory and *constant* computation. (Sutton & Barto, §2.4)

Incremental implementation: the derivation

Given Q_n and the n-th reward R_n, the new average of *all* n rewards is

$$
\begin{aligned}
Q_{n+1} = \frac{1}{n}\sum_{i=1}^{n} R_i \;&=\; \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right) \\
&= \frac{1}{n}\left(R_n + (n-1)\,\frac{1}{n-1}\sum_{i=1}^{n-1} R_i\right) \\
&= \frac{1}{n}\Big(R_n + (n-1)\,Q_n\Big) \;=\; \frac{1}{n}\Big(R_n + n\,Q_n - Q_n\Big) \\
&= Q_n + \frac{1}{n}\Big[\,R_n - Q_n\,\Big]. \qquad (2.3)
\end{aligned}
$$

The manipulation — rewrite the old sum as $(n-1)Q_n$, then split $(n-1)Q_n = nQ_n - Q_n$ — is the prototype for *every* sample-mean recursion in RL. Holds even for $n = 1$ (then $Q_2 = R_1$ for any Q_1); stores only Q_n and n, costing $O(1)$ per step.

The general update rule, used throughout the course

Equation (2.3) is an instance of a form that recurs throughout the entire course:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize}\Big[\text{Target} - \text{OldEstimate}\Big]. \tag{2.4}$$

- $\big[\text{Target} - \text{OldEstimate}\big]$ is an **error** that shrinks as the estimate is moved a *step* toward the (possibly noisy) "Target" — here the target is the new reward R_n.
- The step size *changes from step to step*: processing the n-th reward uses $\frac{1}{n}$. In general we write the step size as α or, state/action-dependently, $\alpha_t(a)$.

The same skeleton reappears with richer "targets": in TD learning the target is $R_{t+1} + \gamma V(S_{t+1})$; in Q-learning it is $R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$. It is established here in the case where the target is R_n itself.

A simple bandit algorithm

Initialize, for $a = 1$ to k:
$Q(a) \leftarrow 0$
$N(a) \leftarrow 0$

Loop forever:
$$A \leftarrow \begin{cases} \arg\max_a Q(a) & \text{with probability } 1-\varepsilon \quad \text{(breaking ties randomly)} \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$
$R \leftarrow bandit(A)$
$N(A) \leftarrow N(A) + 1$
$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}\left[R - Q(A)\right]$

Incrementally-computed sample averages + ε-greedy selection. (Sutton & Barto, Ch. 2)

Nonstationary problems: down-weighting the distant past

Sample averages are appropriate for *stationary* bandits (reward probabilities fixed). But RL problems are often **nonstationary** (the $q_*(a)$ drift over time), and then recent rewards should be weighted more heavily than long-past ones.

One standard modification replaces the $\frac{1}{n}$ step size in (2.3) by a *constant* $\alpha \in (0, 1]$:

$$Q_{n+1} \doteq Q_n + \alpha\big[R_n - Q_n\big]. \tag{2.5}$$

- The update keeps the "New $\leftarrow$ Old $+ \alpha$[Target $-$ Old]" skeleton; only the step size changed.
- The unrolling below shows that Q_{n+1} becomes a weighted average of *all* past rewards *and the initial estimate* Q_1, with weights that decay into the past.

Recall the broken-IID / stationarity "special feature" of RL. (Sutton & Barto, §2.5)

Constant α: the full unrolling

Expand the recursion (2.5), substituting it into itself repeatedly:

$$
\begin{aligned}
Q_{n+1} &= Q_n + \alpha\big[R_n - Q_n\big] \\
&= \alpha R_n + (1-\alpha)\, Q_n \\
&= \alpha R_n + (1-\alpha)\big[\alpha R_{n-1} + (1-\alpha) Q_{n-1}\big] \\
&= \alpha R_n + (1-\alpha)\alpha R_{n-1} + (1-\alpha)^2 Q_{n-1} \\
&= \alpha R_n + (1-\alpha)\alpha R_{n-1} + (1-\alpha)^2 \alpha R_{n-2} + \cdots \\
&\qquad\qquad \cdots + (1-\alpha)^{n-1}\alpha R_1 + (1-\alpha)^n Q_1 \\
&= (1-\alpha)^n\, Q_1 \;+\; \sum_{i=1}^{n} \alpha(1-\alpha)^{\,n-i}\, R_i. \qquad (2.6)
\end{aligned}
$$

Why (2.6) is a weighted average, and its consequences

The weights sum to one. Geometric series with ratio $(1-\alpha)$:

$$(1-\alpha)^n + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} = (1-\alpha)^n + \alpha \frac{1-(1-\alpha)^n}{1-(1-\alpha)} = 1.$$

So (2.6) is a convex average of Q_1 and $R_1, \ldots, R_n$.

- The weight $\alpha(1-\alpha)^{n-i}$ on R_i depends on its *age* $n-i$; since $1-\alpha < 1$ it **decays exponentially** — an *exponential recency-weighted average*.
- If $1-\alpha = 0$ ($\alpha = 1$) all weight goes on the last reward R_n (using $0^0 = 1$).
- For constant $\alpha < 1$ the initial bias $(1-\alpha)^n Q_1$ **never fully vanishes** (it only decays); with $\alpha = 1$, or with $\alpha_n = \frac{1}{n}$, it disappears once the action has been tried.

Step-size conditions for convergence

For a sequence of step sizes $\{\alpha_n(a)\}$, convergence (with probability 1) is assured by the classical stochastic-approximation conditions:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \qquad \text{and} \qquad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty. \tag{2.7}$$

- First condition: steps are large enough to overcome initial conditions and noise.
- Second condition: steps eventually shrink enough to converge.
- $\alpha_n = \frac{1}{n}$ (sample average) satisfies both; constant $\alpha_n = \alpha$ satisfies only the first — and failing the second is exactly what nonstationary tracking requires.

Sequences satisfying both conditions in (2.7) are common in theory but converge slowly and need tuning, so they are seldom used in practice.

An unbiased constant-step-size update (Ex. 2.7)

Constant $\alpha < 1$ tracks nonstationarity but carries a lasting initial bias $(1-\alpha)^n Q_1$. The tracking is kept and the bias removed by the step size

$$\beta_n \doteq \alpha/\bar{o}_n, \qquad \bar{o}_n \doteq \bar{o}_{n-1} + \alpha\,(1-\bar{o}_{n-1}), \quad \bar{o}_0 \doteq 0. \quad \text{(2.8–2.9)}$$

- $\bar{o}_n$ is a *trace* that starts at 0 and converges to 1: unrolling gives $\bar{o}_n = 1-(1-\alpha)^n$.
- At $n=1$: $\bar{o}_1 = \alpha$, so $\beta_1 = \alpha/\alpha = 1$ — the first update *fully overwrites* Q_1, so **no initial-value bias remains**.
- As $n \to \infty$, $\bar{o}_n \to 1$ and $\beta_n \to \alpha$ — recovering the exponential recency weighting that tracks the moving target.

The trace algebra is the same as in the unrolling of (2.5); this machinery is reused throughout RL. (Sutton & Barto, §2.6, Exercise 2.7)

Beyond ε-greedy

Optimistic initial values. Initialize $Q_1(a)$ high (e.g. $+5$). Every action then "disappoints", driving systematic early exploration — a low-cost method, but tied to initial conditions (fails for nonstationary tasks).

Upper-Confidence-Bound (UCB) selection. Prefer actions that are either high-valued *or* uncertain:

$$A_t \doteq \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]. \qquad (2.10)$$

$N_t(a)$ is the number of times a has been chosen so far (if $N_t(a) = 0$, a counts as maximal — so every action is tried once first); $c > 0$ controls exploration. The square-root term is an *uncertainty bonus*: an action tried rarely (small $N_t(a)$) gets a large bonus, so UCB selects it; as it is tried more the bonus shrinks and the estimate $Q_t(a)$ dominates. The $\ln t$ in the numerator maintains a slowly growing incentive to revisit every action (it grows without bound, but ever more slowly). Unlike ε-greedy, exploration here is *directed* — directed by the size of the uncertainty bonus, not chosen at random.

Regret rates of three exploration rules, and one method only named

Measured against the aim stated with the exploration–exploitation dilemma (sublinear regret, $\mathcal{L}_T / T \to 0$):

- **fixed-ε greedy**: explores at rate ε *forever*, so regret grows like $\varepsilon\, T$ — still *linear*, so the aim is not attained;
- **decaying** $\varepsilon_t \to 0$ (at a suitable rate): the regret is sublinear;
- **UCB**: $\mathcal{L}_T = O(\log T)$ — order-optimal, matching the Lai–Robbins lower bound's rate.

Thompson sampling (1933) is only named here: keep a Bayesian posterior over each $q_*(a)$, draw *one sample per arm*, play the best draw — exploration follows from posterior uncertainty. It also attains $O(\log T)$ regret and is widely used in practice (ads, recommenders, A/B testing). It is the standard starting point for reading beyond this course's methods.

Gradient bandits: a policy, not values

Drop value estimates entirely. Keep a numerical **preference** $H_t(a) \in \mathbb{R}$ per action and define a **parameterized policy** by the **soft-max** (Gibbs/Boltzmann) distribution:

$$\pi_t(a) \doteq \Pr\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b=1}^{k} e^{H_t(b)}}. \qquad (2.11)$$

- π_t is a probability distribution ($\pi_t(a) > 0$, $\sum_a \pi_t(a) = 1$): we **sample** $A_t \sim \pi_t$ rather than take an arg max.
- Only *relative* preferences matter — adding a constant to all $H_t(a)$ leaves π_t unchanged. Initially $H_1(a) = 0$, so π_1 is uniform.

Connection to the later policy methods. This is the first *policy-based* method of the course: we learn the policy directly, as differentiable parameters. Exactly the same soft-max "preferences → sampled actions" device reappears in modern policy-gradient algorithms (REINFORCE, actor–critic, PPO) for the full RL problem. (Sutton & Barto, §2.8)

The gradient-bandit update

After selecting A_t and observing R_t, adjust *all* preferences:

$$\begin{aligned} H_{t+1}(A_t) &= H_t(A_t) + \alpha\,(R_t - \bar{R}_t)\,\big(1 - \pi_t(A_t)\big), \\ H_{t+1}(a) &= H_t(a) - \alpha\,(R_t - \bar{R}_t)\,\pi_t(a), \qquad a \neq A_t, \end{aligned} \tag{2.12}$$

where $\alpha > 0$ and $\bar{R}_t$ is the running mean of the rewards *before* step t (excluding R_t itself, so the baseline is already fixed when A_t is chosen — the unbiasedness argument below requires exactly this; including R_t, as in Sutton's experiments, only rescales the step size by $(t-1)/t$). It serves as a **baseline** and is computed incrementally as in §2.4.

- If $R_t > \bar{R}_t$ (better than usual): $\pi_{t+1}(A_t)$ is *increased*, the others decreased. If $R_t < \bar{R}_t$: the reverse. Selected and non-selected actions always move in opposite directions.
- The derivation that follows shows this is **not** an ad-hoc rule: its *expectation* is exactly the gradient of expected reward.

Why (2.12) works: SGA — the plan

Exact gradient ascent on the performance measure $\mathbb{E}[R_t]$ would update

$$H_{t+1}(a) \doteq H_t(a) + \alpha \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)}, \qquad \mathbb{E}[R_t] = \sum_x \pi_t(x)\, q_*(x). \tag{2.13}$$

- We cannot compute this: the $q_*(x)$ are unknown.
- **Plan.** Rewrite the gradient $\partial \mathbb{E}[R_t]/\partial H_t(a)$ as an *expectation* of a quantity we *can* observe each step, then take a single sample of it. The result will be exactly (2.12) — establishing it as **stochastic gradient ascent**.

The derivation has four steps: (1) insert a baseline; (2) form an expectation [tower property]; (3) differentiate the soft-max; (4) assemble.

Step 1 — insert a baseline

Differentiate $\mathbb{E}[R_t] = \sum_x \pi_t(x)\, q_*(x)$ (only π_t depends on H_t):

$$\frac{\partial\, \mathbb{E}[R_t]}{\partial H_t(a)} = \sum_x q_*(x)\, \frac{\partial \pi_t(x)}{\partial H_t(a)} = \sum_x \left(q_*(x) - B_t\right) \frac{\partial \pi_t(x)}{\partial H_t(a)}.$$

- The **baseline** B_t may be *any* scalar independent of x. Subtracting it changes nothing because the gradient sums to zero:

$$\sum_x \frac{\partial \pi_t(x)}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \underbrace{\sum_x \pi_t(x)}_{=1} = 0.$$

- Intuition: as $H_t(a)$ changes, some action probabilities rise and others fall, but the total is always 1 — so $\sum_x B_t\, \partial \pi_t(x)/\partial H_t(a) = 0$.

Step 2 — form an expectation (the tower property)

Multiply and divide each term by $\pi_t(x)$:

$$\frac{\partial\, \mathbb{E}[R_t]}{\partial H_t(a)} = \sum_x \pi_t(x)\,\big(q_*(x) - B_t\big)\,\frac{\partial \pi_t(x)/\partial H_t(a)}{\pi_t(x)}.$$

Since $A_t \sim \pi_t$, the sum $\sum_x \pi_t(x)(\cdots)$ *is* an expectation over A_t:

$$= \mathbb{E}\left[\big(q_*(A_t) - B_t\big)\,\frac{\partial \pi_t(A_t)/\partial H_t(a)}{\pi_t(A_t)}\right].$$

The decisive substitution: R_t for $q_*(A_t)$. They are *not* equal, but their conditional means are: $\mathbb{E}[R_t \mid A_t] = q_*(A_t)$. The **tower property** of conditional expectation then lets us swap them *inside* the outer expectation (see the aside on the tower property).

Aside: the tower property of conditional expectation

Law of total / iterated expectation ("the tower rule"): for any (integrable) random variables X, Y,

$$\mathbb{E}\big[\,X\,\big] \;=\; \mathbb{E}\big[\,\mathbb{E}[\,X \mid Y\,]\,\big].$$

More generally, for a function $g(Y)$ that is fixed once Y is known,

$$\mathbb{E}\big[\,g(Y)\,X\,\big] = \mathbb{E}\big[\,g(Y)\,\mathbb{E}[X \mid Y]\,\big]. \qquad (\star)$$

In our case take $Y = A_t$, $X = R_t$, and $g(A_t) = \dfrac{\partial \pi_t(A_t)/\partial H_t(a)}{\pi_t(A_t)}$ (a function of A_t alone). Because $\mathbb{E}[R_t \mid A_t] = q_*(A_t)$, identity $(\star)$ gives

$$\mathbb{E}[q_*(A_t)\,g(A_t)] = \mathbb{E}[\mathbb{E}[R_t \mid A_t]\,g(A_t)] = \mathbb{E}[R_t\,g(A_t)]\,.$$

Why it is subtle: R_t is random *even after* A_t is fixed, yet conditioning on A_t replaces it by its mean $q_*(A_t)$ with no further assumption. This "condition, average the inside, then average again" pattern recurs throughout RL — it is exactly how policy-gradient estimators stay unbiased.

Step 2 (cont.) — the sampled gradient

Applying the tower property with baseline $B_t = \bar{R}_t$:

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \mathbb{E}\left[(R_t - \bar{R}_t)\,\frac{\partial \pi_t(A_t)/\partial H_t(a)}{\pi_t(A_t)}\right].$$

- The bracketed quantity uses only observed values (R_t, $\bar{R}_t$, π_t) — it can be **sampled on each step**.
- It remains to evaluate the soft-max derivative $\partial \pi_t(A_t)/\partial H_t(a)$ (Step 3).

Step 3 — the soft-max derivative

Claim. $\dfrac{\partial \pi_t(x)}{\partial H_t(a)} = \pi_t(x)\big(\mathbf{1}_{a=x} - \pi_t(a)\big)$, with $\mathbf{1}_{a=x}$ the Kronecker delta.

Proof (quotient rule on $\pi_t(x) = e^{H_t(x)} \big/ \sum_y e^{H_t(y)}$; note $\partial e^{H_t(x)}/\partial H_t(a) = \mathbf{1}_{a=x} e^{H_t(x)}$ and $\partial \sum_y e^{H_t(y)}/\partial H_t(a) = e^{H_t(a)}$):

$$\frac{\partial \pi_t(x)}{\partial H_t(a)} = \frac{\mathbf{1}_{a=x}\, e^{H_t(x)} \sum_y e^{H_t(y)} \;-\; e^{H_t(x)}\, e^{H_t(a)}}{\big(\sum_y e^{H_t(y)}\big)^2}$$

$$= \frac{\mathbf{1}_{a=x}\, e^{H_t(x)}}{\sum_y e^{H_t(y)}} - \frac{e^{H_t(x)}\, e^{H_t(a)}}{\big(\sum_y e^{H_t(y)}\big)^2} = \mathbf{1}_{a=x}\, \pi_t(x) - \pi_t(x)\pi_t(a)$$

$$= \pi_t(x)\big(\mathbf{1}_{a=x} - \pi_t(a)\big). \qquad \square$$

Step 4 — assemble (2.12)

Substitute the soft-max derivative $\partial \pi_t(A_t)/\partial H_t(a) = \pi_t(A_t)\big(\mathbf{1}_{a=A_t} - \pi_t(a)\big)$ into the sampled gradient; the $\pi_t(A_t)$ cancels:

$$\frac{\partial\, \mathbb{E}[R_t]}{\partial H_t(a)} = \mathbb{E}\big[(R_t - \bar{R}_t)\,(\mathbf{1}_{a=A_t} - \pi_t(a))\big].$$

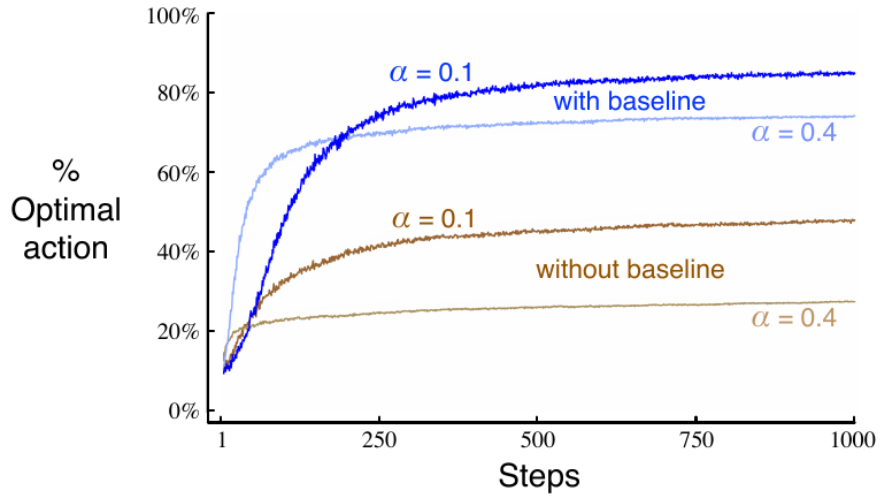
Replacing this expectation by its **one-sample** estimate in (2.13),

$$H_{t+1}(a) = H_t(a) + \alpha\,(R_t - \bar{R}_t)\,(\mathbf{1}_{a=A_t} - \pi_t(a)) \quad \text{for all } a,$$

which is exactly (2.12) (the case $a = A_t$ gives the $1 - \pi_t(A_t)$ term). ■

- **Conclusion:** the gradient bandit is *stochastic gradient ascent* on $\mathbb{E}[R_t]$ — so its convergence results apply when the step sizes satisfy (2.7).
- The baseline B_t does *not* bias the expected update (any x-independent B_t works), but choosing $B_t = \bar{R}_t$ **reduces its variance**, speeding convergence.

Gradient bandit: the effect of the baseline



.5: Average performance of the gradient bandit algorithm with and without a reward
n the 10-armed testbed when the $q_*(a)$ are chosen to be near +4 rather than near zero.

On a testbed with $q_*(a)$ near $+4$: *with* the baseline the method is unaffected by the reward shift; *without* it ($B_t \equiv 0$), the fraction of optimal actions is substantially lower after the first few dozen steps — an instance of why the baseline matters. (Sutton & Barto, Fig. 2.5)

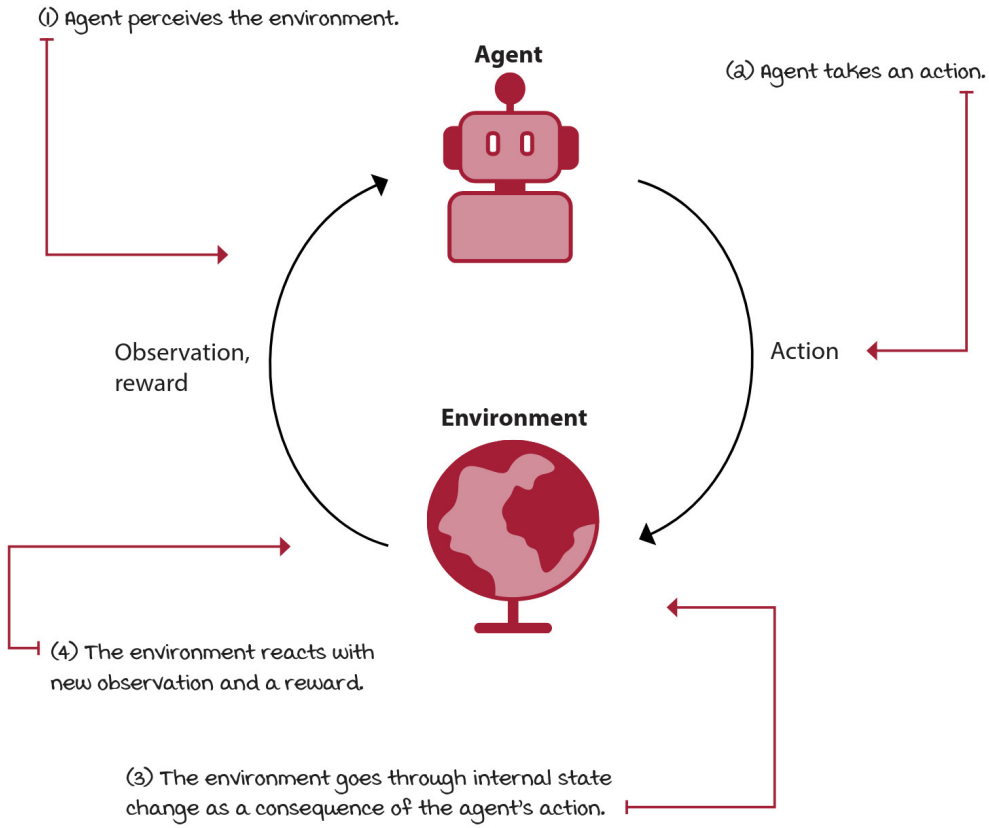
Associative search: the bridge to the full problem

So far the task is **nonassociative**: one situation, find/track a single best action. The general RL problem has *many* situations and the goal is a **policy** — a mapping from situations to actions.

- **Associative search** (*contextual bandits*): at each step a distinctive *context* is observed (e.g. a slot machine changes color), and we learn a policy "if context c, play action a".
- Each action still affects *only the immediate reward* — no consequences for future situations.

Contextual bandits sit *between* the k-armed bandit and full RL. Allow actions to also influence the *next situation*, and we obtain the **Markov Decision Process** — next lecture. (Sutton & Barto, §2.9)

What the bandit is missing



The bandit has a *greedy horizon*: one action, one reward, and the episode ends. The full RL problem adds a **state** that changes, so actions have *delayed* and *sequential* consequences — the agent must assign credit across time.

Next lecture: the Markov Decision Process

and action spaces, and I'll provide you with the discount factors. We'll assume horizons of infinity, and won't need the initial state distribution. Additionally, the most crucial part of this chapter is to give you an awareness of the components of MDPs and POMDPs. Remember, you won't have to do much more building of MDPs than what you've done in this chapter. Nevertheless, let me define MDPs and POMDPs so we're in sync.



SHOW ME THE MATH

MDPs vs. POMDPs

$$MDP(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{S}_\theta, \gamma, \mathcal{H})$$

(1) MDPs have state space $\mathcal{S}$, action space $\mathcal{A}$, transition function $\mathcal{T}$, reward signal $\mathcal{R}$. They also has a set of initial states distribution $\mathcal{S}_\theta$, the discount factor γ, and the horizon $\mathcal{H}$.

$$POMDP(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{S}_\theta, \gamma, \mathcal{H}, \mathcal{O}, \mathcal{E})$$

(2) To define a POMDP, you add the observation space $\mathcal{O}$ and an emission probability $\mathcal{E}$ that defines the probability of showing an observation o given a state s. Very simple.

An MDP is the tuple $(S, A, T, R, S_\theta, \gamma, H)$: state space, actions, transition function, reward signal, initial-state distribution, discount factor, horizon. A POMDP adds an observation space O and emission probabilities E. Each component is developed in Lecture 2. (Morales, Ch. 2)

Summary — Lecture 1: from bandit objects to RL objects

The lecture introduced these ideas in their simplest (state-free) form:

- Evaluative feedback was contrasted with instructive feedback and shown to force exploration.
- The k-armed bandit was formalized: $R_t \sim \nu_{A_t}$, value $q_*(a) = \mathbb{E}[R_t \mid A_t{=}a]$, optimum v_*, a_*, and **regret** $\mathcal{L}_T$.
- Action-value estimation was developed via the sample average (2.1), with greedy, ε-greedy, and UCB (2.10) selection.
- The incremental update (2.3) was generalized to the universal rule New $\leftarrow$ Old $+ \alpha$[Target $-$ Old], with constant-α recency weighting (2.6) and convergence conditions (2.7).
- Gradient bandits supplied a soft-max *policy* (2.11) that is *sampled*, with update (2.12) derived as stochastic gradient ascent — the course's first *policy-based* method, using the **tower property** for an unbiased gradient.

Next: add a *state* and a *transition function*, turn the reward into a *return*, and the bandit becomes a Markov Decision Process.